Tutorial Note 1

Cyber Differential Games, PMP, RL/DRL, MADRL, Neural ODE, PINN/PIDL, and Hybrid Control

A self-contained guide from models to implementation

Prepared for advanced research discussion and supervision

Contents

1. How This Guide Is Organized

The note is organized around a clean workflow:

$$\boxed{\text{cyber dynamics}} \rightarrow \boxed{\text{control/game objective}} \rightarrow \boxed{\text{continuous-time theory}} \rightarrow \boxed{\text{learning formulation}} \rightarrow \boxed{\text{implementation and evaluation}} \tag{1}$$

The most important improvement is that RL, DRL, and MARL are not presented as unrelated tools after PMP. Instead, the note explains exactly how a continuous-time optimal control or differential game is converted into an MDP or Markov game, and what information from PMP is preserved or lost in that conversion.

Key methodological principle. PMP and differential games define the continuous-time mathematical problem. RL/DRL/MARL usually solve a discretized feedback-control problem derived from that problem. PINN/PIDL can either learn unknown dynamics, solve the PMP residual system, or approximate HJB/HJI equations. These are different levels of coupling, and they should not be confused.

2. Notation and Method Selection

This note uses a small number of repeated symbols. The same notation is used across optimal control, differential games, RL, and PINN/PIDL so that the link among methods is visible.

Symbol	Meaning
$x(t)$	continuous-time cyber state, for example malware compartments, APT asset states, or misinformation populations
$u_D(t), u_A(t)$	continuous-time defender and attacker controls
a_D^k, a_A^k	discrete-time actions selected at decision epoch t_k
t_k	action/observation point in the sampled MDP or Markov game after conversion
$\Delta t_k = t_{k+1} - t_k$	action interval; fixed Δt is a special case, not a requirement
τ_j	impulse or event point already present in the original continuous or hybrid model
θ	physical or propagation parameters, such as infection, repair, forgetting, deception-decay, or correction rates
J_D, J_A	cumulative defender and attacker objectives in continuous time
H	Hamiltonian used by PMP or differential-game optimality conditions
$\lambda(t)$	costate or adjoint variable; it measures marginal value of changing the state
s_k	RL/DRL/MARL state at decision epoch t_k; usually derived from $x(t_k)$ and observations
r_k	reward over the interval $[t_k, t_{k+1}]$, usually negative running cost
π	policy mapping states to actions, e.g., $\pi_D(a_D \mid s)$

2.1 What each method consumes and outputs

A useful way to avoid confusion is to ask what each method consumes as input and what it returns as output.

Method	Main input	Main output
PMP	ODE model, cost functional, constraints	optimality system: state, costate, stationarity, boundary conditions
FBSM	PMP optimality system and initial guess for controls	open-loop trajectory $u^*(t)$ and state $x^*(t)$
RL/DQN	MDP simulator (s, a, r, s') with discrete actions	feedback policy or Q-function $Q(s, a)$
PPO/SAC/DDPG	simulator with continuous or stochastic actions	feedback actor $\pi(a\|s)$ and critic
MARL	Markov game simulator with multiple agents	attacker and defender policies, often with centralized critics
Neural ODE	observed trajectories and differentiable ODE solver	learned dynamics $\dot{x} = f_\theta(x, u, t)$ or residual term g_θ
PINN/PIDL	data, equations, constraints, priors, possibly PMP/HJI residuals	states, parameters, controls, costates, or value functions satisfying residual constraints

Practical selection rule. Use PMP/FBSM when the model is known, continuous, and low- to moderate-dimensional. Use RL/DRL when actions are operational decisions and feedback policies are needed. Use MARL when the attacker adapts. Use Neural ODE when the dynamics are partly unknown. Use PINN/PIDL when equations and constraints are important and data are sparse, or when one wants a neural solver for PMP/HJB/HJI residual systems.

3. Game Concepts Before Learning: What Exactly Is Being Solved?

A source of confusion in recent learning-for-differential-games papers is that the word "game" may refer to several different mathematical objects. Before choosing PMP, FBSM, RL, DRL, MARL, MADRL, PINN, or PIDL, it is useful to identify the game type and the solution concept. Differential games extend optimal control from one decision maker to multiple strategic decision makers: each player controls the same evolving state, but each player evaluates the outcome through its own objective. This point is emphasized in classical dynamic-game theory [31, 32] and in cyber differential-game tutorials.

3.1 A compact taxonomy for cyber differential games

The following table is intended as a practical checklist for malware, APT, deception, and propaganda-war models.

Dimension	Meaning	Typical cyber interpretation

Zero-sum vs general-sum	In a zero-sum game, $J_A = -J_D$. In a general-sum game, each player has a different payoff.	Malware attacker-defender games may be approximated as zero-sum. Propaganda, compliance, and insider-assisted APT games are often general-sum because costs, reputational effects, and operational benefits are not exact negatives.
Deterministic vs stochastic	The state equation is an ODE or a stochastic process.	Most propagation models use deterministic ODEs for tractability. RL/-MADRL can add stochasticity through random initial states, random attacks, noisy observations, or network sampling.
Nash vs Stackelberg	In Nash games, players decide simultaneously. In Stackelberg games, a leader commits first and a follower responds.	Attacker-defender malware control is often modeled as Nash. Security investment, auditing, and compliance guidance can be Stackelberg if the organization commits first and employees or attackers respond.
Open-loop vs feedback	Open-loop controls are functions of time. Feedback controls are functions of the observed state.	PMP/FBSM usually returns $u_i(t)$ for a fixed initial state. RL/MADRL usually returns $\pi_i(a_i\|s)$ that adapts to the current state.
Continuous vs impulsive vs hybrid	Continuous controls act through $\dot{x} = f(\cdot)$. Impulses create jumps $x(\tau^+) = G(x(\tau^-), v)$. Hybrid models combine continuous flows, discrete modes, and impulses.	Continuous propaganda spending, continuous patching, or monitoring affect transition rates. Human incident response, media-literacy campaigns, isolation, and takedown actions may be better modeled as impulses.

Why this taxonomy matters for learning. DQN, PPO, SAC, MADDPG, MAPPO, and other MADRL methods do not directly require the original PMP equations. They require an environment. The game taxonomy determines what the environment should contain: one agent or multiple agents; zero-sum or general-sum rewards; simultaneous or sequential actions; discrete, continuous, or hybrid actions; continuous flow only or impulse jumps.

3.2 Nash equilibrium in continuous time and in Markov games

For a two-player continuous-time differential game,

$$\dot{x}(t) = f(x(t), u_1(t), u_2(t), t), \quad x(0) = x_0, \tag{2}$$

with objectives $J_1(u_1, u_2)$ and $J_2(u_1, u_2)$, an open-loop Nash equilibrium (u_1^*, u_2^*) satisfies

$$J_1(u_1^*, u_2^*) \geq J_1(u_1, u_2^*) \quad \text{for all } u_1, \tag{3}$$

$$J_2(u_1^*, u_2^*) \geq J_2(u_1^*, u_2) \quad \text{for all } u_2, \tag{4}$$

when the convention is payoff maximization. If the problem is written as cost minimization, the inequality directions are reversed. PMP gives necessary conditions for such equilibria by introducing one Hamiltonian and one costate system for each player. In nonzero-sum games these conditions are normally necessary rather than sufficient, so numerical comparison against alternative strategies remains important.

After time discretization, state observation, action selection, and ODE/impulse simulation, the same strategic idea becomes a Markov game:

$$\mathcal{G} = \big(\mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i\in\mathcal{N}}, P, \{r_i\}_{i\in\mathcal{N}}, \gamma\big). \tag{5}$$

Here $\mathcal{N}$ is the player set, $\mathcal{S}$ is the state space, $\mathcal{A}_i$ is player i's action space, $P(s'|s, a_1, \ldots, a_N)$ is the transition kernel, r_i is the reward of player i, and γ is a discount factor. A Markov-perfect Nash equilibrium is a profile of feedback policies $\pi_i^*(a_i|s)$ such that no player can improve its value by unilateral deviation:

$$V_i^{\pi_i^*, \pi_{-i}^*}(s) \geq V_i^{\pi_i, \pi_{-i}^*}(s), \quad \forall \pi_i, \ \forall s. \tag{6}$$

This is the precise object that MARL and MADRL try to approximate.

3.3 How the HODGE propaganda-war paper fits this taxonomy

The HODGE/ORIENTATION framework is a useful example because it is not a simple continuous-control game. It combines a degree-based network model for dual competitive information spreading with a dual hybrid control mechanism: continuous-time propaganda investment and discrete-time impulsive counter-propaganda investment [18]. In words, propaganda is modeled as persistent influence over an interval, while counter-propaganda is modeled as targeted interventions at selected moments. This distinction is exactly the distinction between continuous flow control and impulse/jump control.

A simplified abstraction is

$$\dot{x}(t) = f(x(t), u_R(t), u_B(t)), \quad t \notin \tau_R \cup \tau_B, \tag{7}$$

$$x(\tau_i^+) = G_R(x(\tau_i^-), v_R(\tau_i)) \quad \text{or} \quad x(\tau_i^+) = G_B(x(\tau_i^-), v_B(\tau_i)), \tag{8}$$

where u_R, u_B are continuous propaganda investment rates and v_R, v_B are impulsive counter-propaganda investment levels. The original HODGE algorithm approximates a Nash strategy profile by iteratively updating continuous and impulsive decisions. A learning-based extension would not simply "replace PMP by MADRL." Instead, it would define a Markov game in which each party observes a state summary, selects continuous and/or impulsive investment actions at decision epochs, applies jump maps if an impulse is selected, integrates the degree-based ODE until the next decision epoch, and receives a reward based on narrative benefit minus investment cost.

Hybrid propaganda game as a Markov game

Observe degree-based state summary s_k

$\Downarrow$

Red chooses (m_R^k, v_R^k, u_R^k), Blue chooses (m_B^k, v_B^k, u_B^k)

$\Downarrow$

If m_i^k is impulsive, apply jump map G_i

$\Downarrow$

Integrate ODE with continuous investments over $[t_k, t_{k+1}]$

$\Downarrow$

Compute rewards: influence benefit − continuous cost − impulse cost

$\Downarrow$

Update MADRL policies by self-play or CTDE

4. Cyber Dynamic Models: What Is Being Controlled?

4.1 Controlled malware propagation

A basic malware or virus propagation model is

$$\dot{S}(t) = -\beta S(t)I(t) - u_p(t)S(t) + \omega R(t), \tag{9}$$

$$\dot{I}(t) = \beta S(t)I(t) - \gamma I(t) - u_c(t)I(t), \tag{10}$$

$$\dot{R}(t) = \gamma I(t) + u_p(t)S(t) + u_c(t)I(t) - \omega R(t). \tag{11}$$

Here $S(t)$ denotes vulnerable devices, $I(t)$ compromised devices, and $R(t)$ recovered or protected devices. The infection term βSI expresses the idea that vulnerable devices are compromised through interaction with infected devices. The control $u_p(t)$ represents patching or hardening, and $u_c(t)$ represents cleaning, monitoring, or incident response.

This model is suitable for introducing optimal control because the defender must balance two competing goals: reduce $I(t)$ and avoid excessive control cost. A typical objective is

$$J_D(u_p, u_c) = \int_0^T \left[A_I I(t) + \frac{B_p}{2} u_p^2(t) + \frac{B_c}{2} u_c^2(t) \right] \mathrm{d}t + A_T I(T). \tag{12}$$

The term $A_I I(t)$ is security loss, while $B_p u_p^2/2$ and $B_c u_c^2/2$ penalize aggressive intervention. The terminal term $A_T I(T)$ penalizes residual compromise.

4.2 APT repair and deception dynamics

APT attacks are persistent and stealthy, so it is useful to include repair, isolation, and deception. A compact APT-oriented model is

$$\dot{V} = -\beta(1 - \chi\eta_k)VC - u_h V + \omega P, \tag{13}$$

$$\dot{C} = \beta(1 - \chi\eta_k)VC - (\gamma + u_r + u_i)C, \tag{14}$$

$$\dot{P} = u_h V + (\gamma + u_r + u_i)C - \omega P. \tag{15}$$

Here V is vulnerable but uncompromised assets, C is compromised assets, and P is protected or repaired assets. The action $\eta_k \in [0, 1]$ is chosen at decision epoch t_k and held or mapped over the next interval. The factor $(1 - \chi\eta_k)$ reduces the effective compromise rate during that interval. In cyber terms, honeypots, decoys, moving-target defense, or false information reduce useful attacker contact without introducing a fourth core compartment. This extends differential-game APT repair ideas such as Yang et al. [17] and connects naturally with impulsive and hybrid APT defense models [19, 20].

4.3 Rumor and misinformation defense

For rumor, misinformation, or cyber propaganda, a simple correction model is

$$\dot{U} = -\beta_M UM - \beta_C UC, \tag{16}$$
$$\dot{M} = \beta_M UM - \alpha MC - \delta M - u_f M, \tag{17}$$
$$\dot{C} = \beta_C UC + \alpha MC + u_f M - \delta_C C, \tag{18}$$
$$\dot{R} = \delta M + \delta_C C. \tag{19}$$

U is unaware users, M misinformation believers or spreaders, C correction believers or counter-message spreaders, and R inactive or removed users. The control $u_f(t)$ represents fact-checking, trusted correction, platform intervention, or public communication. This model makes sense for cyber propaganda games, where opposing sides can strategically influence narrative populations [18].

Common structure. Malware, APT, and misinformation models all have the form

$$\dot{x} = f(x, u_D, u_A, \theta, t),$$

where x is a cyber/social state, u_D is defense intervention, u_A is attack effort, and θ contains propagation parameters. This common structure is why PMP, FBSM, HJB/HJI, RL/DRL/MARL, Neural ODE, and PINN/PIDL can be studied in one framework.

4.4 How to read these models operationally

The compartment variables should not be interpreted as only biological analogies. In cybersecurity, a compartment can represent a proportion of devices, accounts, hosts, services, narratives, or users. For example, $I(t)$ in a malware model can be the fraction of compromised IoT devices, while $M(t)$ in a misinformation model can be the fraction of users currently spreading a false narrative. A control is any costly intervention that changes transition rates. Patching changes the transition from vulnerable to protected. Cleaning changes the transition from compromised to recovered. Fact-checking changes the transition from misinformation to correction. Deception changes the effective attack rate by altering the attacker's information.

The main modeling decision is therefore not the name of the compartments but the transition mechanism. A paper should explain why each term exists. For example, βSI means mass-action infection: compromise increases when both vulnerable and infected assets are present. The term $(1 - \chi\eta_k)$ means a deception action reduces useful attacker contact over a decision interval. The term $u_f M$ means correction effort converts misinformation spreaders into corrected or inactive users. Clear interpretation of every term is essential before applying PMP or learning methods.

4.5 What makes a cyber model suitable for learning-based control?

A model is suitable when it can be simulated under candidate actions. For RL/DRL/MARL, the simulator may be an ODE solver, an agent-based simulator, a cyber range, or a hybrid environment. For PINN/PIDL, the model must provide differentiable residuals or differentiable approximations. For Neural ODE, one needs trajectory data or simulated traces to train the dynamics. For PMP/FBSM, the model should be differentiable with respect to states and controls so that H_x, H_u, and costate equations can be derived.

5. Continuous, Discrete, Impulsive, and Hybrid Cyber Control

This section clarifies a point that often causes confusion. In cyber control papers, the words *continuous*, *discrete*, *impulsive*, and *hybrid* refer to different parts of the model. They should not be used interchangeably.

5.1 Continuous-time state dynamics and continuous control

A continuous-time cyber model assumes that the state changes continuously over time:

$$\dot{x}(t) = f(x(t), u(t), t), \qquad t \in [0, T]. \tag{20}$$

Here $x(t)$ may be the proportion of vulnerable, infected, compromised, recovered, or misinformed users. A continuous control $u(t)$ is a function defined over time. For example,

$$u(t) = \text{patching intensity at time } t, \qquad 0 \le u(t) \le 1. \tag{21}$$

This means that the defender can continuously adjust the strength of patching, monitoring, deception, or fact-checking. Mathematically, $u(t)$ may be an open-loop function of time or a feedback law $u(t) = \pi(x(t), t)$.

Important distinction. Continuous control does not necessarily mean the defender manually changes the action at every instant. It means the mathematical decision variable is a function over the whole time interval. In numerical implementation, it is often approximated by a piecewise-constant or piecewise-linear function.

For example, in a malware model,

$$\dot{S} = -\beta SI - u_p(t)S, \qquad \dot{I} = \beta SI - (\gamma + u_c(t))I, \tag{22}$$

where $u_p(t)$ is continuous patching intensity and $u_c(t)$ is continuous cleaning intensity. PMP/FBSM is natural for this setting because the unknown is a control trajectory $u(t)$ over $[0, T]$.

5.2 Discrete decision with continuous flow: no jump

In real cyber operations, decisions are usually made at discrete times:

$$0 = t_0 < t_1 < \cdots < t_K = T, \qquad \Delta t_k = t_{k+1} - t_k. \tag{23}$$

The intervals may be fixed, event-triggered, or risk-triggered. At time t_k, the defender chooses an action a_k, such as `patch`, `monitor`, `isolate`, or `deceive`. After that choice, the system still evolves continuously until the next decision time:

$$\dot{x}(t) = f(x(t), a_k, t), \qquad t \in [t_k, t_{k+1}). \tag{24}$$

There is no instantaneous jump in the state:

$$x(t_k^+) = x(t_k^-). \tag{25}$$

Only the vector field changes. For instance, choosing `patch` may increase the patching rate during the next day, but the number of infected machines does not instantly drop at the exact decision instant.

Discrete decision without jump, zero-order hold example:

observe $x_k \to$ choose $a_k \to$ apply action map over $[t_k, t_{k+1}) \to$ integrate ODE $\to$ obtain x_{k+1}

This is the cleanest bridge to RL/DRL. The induced MDP transition is

$$x_{k+1} = \text{ODESolve}\left(f(\cdot, a_k), x_k, t_k, t_{k+1}\right). \tag{26}$$

The reward is usually the negative accumulated cost over the same interval:

$$r_k = -\int_{t_k}^{t_{k+1}} L(x(t), a_k)\, \mathrm{d}t. \tag{27}$$

DQN/DDQN can be used if a_k is discrete; PPO/SAC/DDPG can be used if the action includes continuous intensity.

5.3 Impulsive control: discrete decision with state jump

An impulsive model assumes that some interventions cause an instantaneous state change at selected times. The dynamics are then

$$\begin{cases} \dot{x}(t) = f(x(t), u(t), t), & t \neq \tau_j, \\ x(\tau_j^+) = G_j(x(\tau_j^-), a_j), & t = \tau_j. \end{cases} \tag{28}$$

Here τ_j is an impulse time, $x(\tau_j^-)$ is the state just before the impulse, and $x(\tau_j^+)$ is the state just after the impulse. In cybersecurity, impulses are natural when a decision has an immediate effect, for example:

- emergency patching instantly moves a fraction of vulnerable machines to protected machines;
- isolation instantly removes a fraction of compromised hosts from the active network;
- a takedown operation instantly removes malicious accounts or command-and-control nodes;
- a fact-checking campaign instantly shifts a fraction of misinformed users into a corrected class.

A simple malware impulse can be written as

$$\begin{bmatrix} S(\tau_j^+) \\ I(\tau_j^+) \\ R(\tau_j^+) \end{bmatrix} = \begin{bmatrix} (1-p_j)S(\tau_j^-) \\ (1-q_j)I(\tau_j^-) \\ R(\tau_j^-) + p_j S(\tau_j^-) + q_j I(\tau_j^-) \end{bmatrix}, \tag{29}$$

where p_j is the emergency patching fraction and q_j is the emergency isolation/cleaning fraction. Impulsive APT defense and defense outsourcing models follow this logic [19, 20].

Impulsive RL transition:

observe pre-jump state $x_k^- \to$ choose impulse action $a_k \to$ jump $y_k = G(x_k^-, a_k) \to$ integrate ODE $\to$ next state x_{k+1}^-

The MDP transition becomes

$$y_k = G(x_k, a_k), \qquad x_{k+1} = \text{ODESolve}(f, y_k, t_k, t_{k+1}). \tag{30}$$

The reward should include both the running cost and the impulse cost:

$$r_k = -\int_{t_k}^{t_{k+1}} L(x(t), a_k)\, \mathrm{d}t - C_{\mathrm{imp}}(x_k, a_k). \tag{31}$$

Thus the RL environment must implement both a `jump()` function and an `integrate()` function.

5.4 Hybrid control: discrete mode plus continuous intensity

Hybrid cyber actions combine a discrete mode and a continuous parameter:

$$a_k = (m_k, v_k), \qquad m_k \in \{1, \ldots, M\}, \quad v_k \in [0, 1]. \tag{32}$$

For example,

$$m_k = \texttt{patch}, \qquad v_k = 0.7 \tag{33}$$

means that the defender chooses the patching mode with intensity 0.7. Hybrid control may affect the system in two ways:

1. **Mode-dependent continuous flow:**

$$\dot{x} = f_{m_k}(x, v_k, t), \qquad t \in [t_k, t_{k+1}). \tag{34}$$

2. **Mode-dependent jump plus flow:**

$$y_k = G_{m_k}(x_k, v_k), \qquad x_{k+1} = \mathrm{ODESolve}(f_{m_k}, y_k, t_k, t_{k+1}). \tag{35}$$

This form is often the most realistic for cyber defense because the defender chooses both *what to do* and *how strongly to do it*.

5.5 Mixed continuous-discrete differential games

In an attacker-defender game, both players may have hybrid actions:

$$a_D^k = (m_D^k, v_D^k), \qquad a_A^k = (m_A^k, v_A^k). \tag{36}$$

The continuous flow can be written as

$$\dot{x} = f_{m_D^k, m_A^k}(x, v_D^k, v_A^k, t), \qquad t \in [t_k, t_{k+1}). \tag{37}$$

If the actions also cause jumps,

$$y_k = G_{m_D^k, m_A^k}(x_k, v_D^k, v_A^k), \qquad x_{k+1} = \mathrm{ODESolve}(f_{m_D^k, m_A^k}, y_k, t_k, t_{k+1}). \tag{38}$$

The induced Markov game has transition

$$P(s_{k+1} | s_k, a_D^k, a_A^k) \tag{39}$$

implemented by jump and ODE integration. Defender and attacker rewards are

$$r_D^k = -\int_{t_k}^{t_{k+1}} L_D(x(t), a_D^k, a_A^k)\,\mathrm{d}t - C_D^{\mathrm{imp}}, \tag{40}$$

$$r_A^k = \int_{t_k}^{t_{k+1}} L_A(x(t), a_D^k, a_A^k)\,\mathrm{d}t - C_A^{\mathrm{imp}}. \tag{41}$$

This is the natural setting for MARL. The continuous-time model still provides the scientific mechanism, but the learning algorithm interacts with a discrete-time Markov game simulator.

5.6 Which method should be used?

Model type	Mathematical form	Suitable learning/control method
Continuous control	$\dot{x} = f(x, u(t), t)$	PMP, FBSM, direct collocation, PMP-informed PINN
Discrete decision without jump	$\dot{x} = f(x, a_k, t)$ over each interval	DQN/DDQN for discrete actions; PPO/SAC/DDPG for continuous actions
Impulsive control	$x^+ = G(x^-, a_k)$ plus continuous flow	impulse control theory; RL with jump+ODE transition; hybrid PMP for baseline
Hybrid action	$a_k = (m_k, v_k)$	parameterized-action RL, hybrid PPO/SAC, relaxed PINN for differentiable approximation
Attacker-defender hybrid game	$\dot{x} = f(x, a_D^k, a_A^k)$ and possibly jumps	Markov game, MARL, CTDE, MAPPO/MADDPG-style training

6. From Continuous-Time Control to MDPs and Markov Games

This section is the bridge that is often missing in papers. It explains how a continuous-time optimal control or differential game becomes a discrete-time learning environment.

6.1 Original continuous-time optimal control problem

A defender-only control problem is

$$\min_{u_D(\cdot)} \quad J_D = \Phi(x(T)) + \int_0^T L_D(x(t), u_D(t), t)\,\mathrm{d}t, \tag{42}$$

$$\text{s.t.} \quad \dot{x}(t) = f(x(t), u_D(t), t), \qquad x(0) = x_0. \tag{43}$$

PMP and FBSM solve this problem directly in continuous time. RL and DRL usually solve a derived discrete-time feedback-control problem.

6.2 Does learning always require discretization?

No. There are several different meanings of "discretization," and confusing them leads to unclear papers.

Object	What is discretized	Is it required?
Continuous-time PMP/F-BSM	A numerical mesh is used to solve ODE and costate equations.	Required for computation, but it is not an MDP.
Model-free RL or MARL	The learner receives transitions at action points t_k.	Usually required because replay buffers and Bellman updates use (s_k, a_k, r_k, s_{k+1}).
PINN, PIDL, Neural ODE	Collocation points or observed times are sampled.	Required for training data or residuals, but the learned object may still represent continuous time.

Therefore, learning-based methods do not always require turning the original model into a simple discrete-time MDP. Model-free DQN, PPO, SAC, and MADRL usually do. PINN/PIDL and Neural ODE can keep a continuous-time residual and train on sampled points. FBSM uses a mesh for numerical integration but still solves the continuous-time optimality system.

Piecewise constant is common, not mandatory. When an RL action is chosen at t_k, the simplest implementation is zero-order hold:

$$u(t) = \Gamma(a_k), \qquad t \in [t_k, t_{k+1}).$$

This makes the applied control piecewise constant on each action interval. It is common because it gives a clean environment transition. It is not required by the original differential game. One may use piecewise-linear controls, event-triggered impulses, continuous actor outputs inside a differentiable ODE solver, or a feedback law $u(t) = \pi(x(t), t)$ evaluated continuously. If a paper uses zero-order hold, it should say so explicitly.

6.3 Decision epochs, impulse times, and nonuniform intervals

Use separate symbols for two different kinds of time points:

$$0 = t_0 < t_1 < \cdots < t_K = T, \qquad \Delta t_k = t_{k+1} - t_k,$$

where t_k is a learning action/observation point after conversion to an MDP or Markov game. The intervals Δt_k do *not* need to be equal. A fixed grid $t_k = k\Delta t$ is only the simplest implementation.

If the original continuous or hybrid model already has impulse or event points, denote them by τ_j. These points belong to the original model, not automatically to the RL conversion.

Relationship	Meaning	Implementation choice
$\tau_j = t_k$ for selected j, k	An original impulse event coincides with a learning action point.	Expose the impulse as an available action or mode at that epoch.
$\{\tau_j\} \subset \{t_k\}$	Only some decision epochs allow emergency actions.	Add an action mask or time-dependent action set.

$\tau_j \in (t_k, t_{k+1})$	The original model has an event inside a learning transition.	Apply the event inside the simulator transition and include its effect in s_{k+1} and r_k.
τ_j chosen by policy	The action includes event timing.	Use a semi-MDP, event-triggered policy, or hybrid action with a waiting time.

In this repository, the default code convention is simple: the policy observes $x(t_k^-)$, chooses an action, the environment applies any jump to obtain $x(t_k^+)$, then it integrates the ODE to the next observation $x(t_{k+1}^-)$. This is the convention used in `cyber_hybrid_env.py`. The code uses fixed `EnvConfig.dt`; the mathematics above allows nonuniform action schedules.

6.4 Step-by-step conversion to an MDP

At each t_k, the defender observes the pre-jump state summary

$$s_k = \psi(x(t_k^-), b_k, o_k), \tag{44}$$

where b_k may represent budget and o_k may represent observations such as alerts or detection scores. The defender chooses an action a_k. The action can be discrete, continuous, or hybrid:

$$\text{discrete: } a_k \in \{\text{patch, monitor, isolate, deceive}\}, \tag{45}$$

$$\text{continuous: } a_k = [u_p^k, u_c^k, u_d^k] \in [0,1]^3, \tag{46}$$

$$\text{hybrid: } a_k = (m_k, v_k), \quad m_k \in \{\text{patch, monitor, deceive}\},\ v_k \in [0,1]. \tag{47}$$

If a_k has an impulse component, the environment first applies a jump map

$$y_k = x(t_k^+) = G(x(t_k^-), a_k). \tag{48}$$

If there is no impulse component, then $y_k = x(t_k^-)$. The continuous dynamics then run between decision points:

$$x_{k+1}^- = \text{ODESolve}\left(f, y_k, a_k, [t_k, t_{k+1}]\right). \tag{49}$$

The reward is the negative of the interval cost:

$$r_k = -C_{\text{jump}}(x(t_k^-), a_k) - \int_{t_k}^{t_{k+1}} L_D(x(t), a_k, t)\,\mathrm{d}t. \tag{50}$$

A simple numerical approximation is

$$r_k \approx -C_{\text{jump}}(x(t_k^-), a_k) - \Delta t_k\, L_D(x_{k+1}^-, a_k, t_{k+1}). \tag{51}$$

Now the problem is an MDP

$$(\mathcal{S}, \mathcal{A}, P, r, \gamma_k), \tag{52}$$

where P is generated by the ODE solver, jump map, and stochastic disturbances. This is the correct Level 1 bridge from optimal control to RL [4].

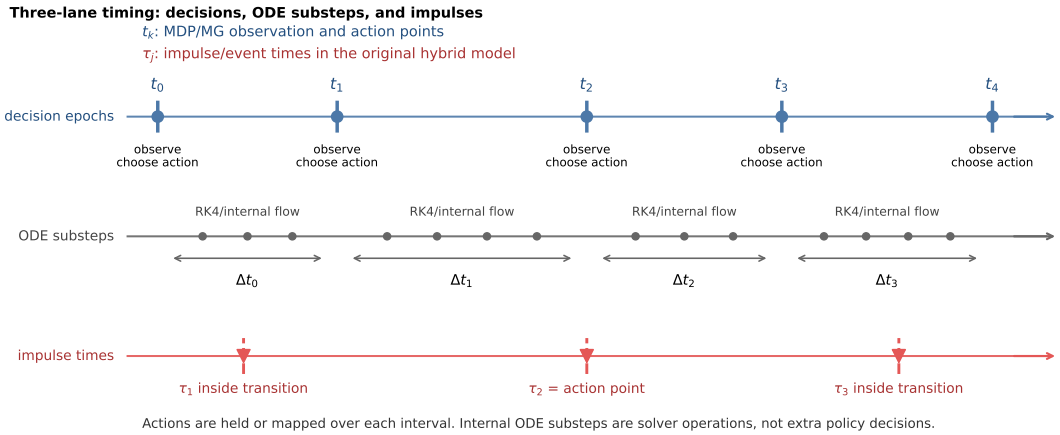

Figure 1: Three-lane action timing used in the sampled-data learning code. Decision epochs t_k are the points where the MDP or Markov game observes the state and chooses actions. ODE substeps are internal numerical integration steps. Impulse times τ_j belong to the original hybrid model and may coincide with, or fall between, decision epochs.

Continuous or hybrid model to MDP

$\dot{x} = f(x, u, t)$ and optional original impulses at τ_j $\Longrightarrow$ choose action points t_k $\Longrightarrow$ observe $s_k = \psi(x(t_k^-))$ $\Longrightarrow$ apply jumps/events, integrate flow, return s_{k+1} $\Longrightarrow$ reward $r_k \approx -C_{\text{jump}} - \int L\,\mathrm{d}t$ $\Longrightarrow$ train RL/DRL policy $\pi(a|s)$.

6.5 Discounting and continuous-time costs

If the original continuous-time objective uses discounting

$$J = \int_0^T e^{-\rho t} L(x(t), u(t))\,\mathrm{d}t, \tag{53}$$

then the interval-specific discrete discount can be chosen as

$$\gamma_k = e^{-\rho \Delta t_k}. \tag{54}$$

For a fixed grid this becomes $\gamma_d = e^{-\rho \Delta t}$. If the original problem has a finite horizon and no discount, many implementations set γ_d close to one and terminate at K. The key is to make the discrete reward consistent with the original running cost. Otherwise, the DRL policy may optimize a different problem from the one stated in the differential-game model.

6.6 Action-to-control map

A common source of confusion is the difference between a discrete RL action and a continuous control in the ODE. The bridge is an action-to-control map

$$\Gamma : \mathcal{A} \to \mathcal{U}, \qquad u(t) = \Gamma(a_k), \quad t \in [t_k, t_{k+1}). \tag{55}$$

For example,

$$\Gamma(\text{patch}) = (u_p, u_c, u_d) = (0.8, 0.1, 0), \tag{56}$$

while

$$\Gamma(\text{deceive}) = (0.1, 0.1, 0.9). \tag{57}$$

In hybrid-action RL, $a_k = (m_k, v_k)$ and the map becomes

$$\Gamma(m_k, v_k) = v_k \Gamma_{max}(m_k), \tag{58}$$

so the agent chooses both intervention type and intensity.

6.7 A practical environment interface

In code, the continuous-time model becomes an environment with two important functions:

ODE-based cyber MDP interface.

1. `reset`: sample or set x_0, budget b_0, attack scenario, and horizon K.
2. `observe`: return $s_k = \psi(x_k, o_k, b_k)$, where o_k may be noisy alerts or partial measurements.
3. `step(a_k)`: convert a_k to ODE controls, integrate the dynamics for one interval, compute reward, update budget, and return $(s_{k+1}, r_k, done)$.

The Markov property requires that s_k contain enough information to predict the next state distribution given the action. If the defender observes only alerts rather than true compromise levels, the problem is closer to a partially observable MDP. A practical workaround is to include a history window, recurrent neural network, or state estimator.

6.8 Conversion to a Markov game

For an attacker-defender differential game,

$$\dot{x} = f(x, u_A, u_D, t), \tag{59}$$

$$J_D = \Phi_D(x(T)) + \int_0^T L_D(x, u_A, u_D, t)\, \mathrm{d}t, \tag{60}$$

$$J_A = \Phi_A(x(T)) + \int_0^T L_A(x, u_A, u_D, t)\, \mathrm{d}t. \tag{61}$$

Choose decision times. At step k, each player observes the state summary before any new jump:

$$a_A^k \sim \pi_A(\cdot|s_k), \qquad a_D^k \sim \pi_D(\cdot|s_k). \tag{62}$$

The joint action may include impulse modes and continuous-flow modes. The transition is

$$x(t_k^+) = G(x(t_k^-), a_A^k, a_D^k), \qquad s_{k+1} = \psi\Big(\text{ODESolve}(f, x(t_k^+), a_A^k, a_D^k, [t_k, t_{k+1}))\Big), \tag{63}$$

and rewards are

$$r_D^k \approx -\Delta t_k\, L_D(s_{k+1}, a_A^k, a_D^k), \tag{64}$$

$$r_A^k \approx \Delta t_k\, L_A(s_{k+1}, a_A^k, a_D^k). \tag{65}$$

This gives a Markov game

$$(\mathcal{S}, \mathcal{A}_A, \mathcal{A}_D, P, r_A, r_D, \gamma_d). \tag{66}$$

MARL learns policies π_A and π_D by repeated interaction.

6.9 What is preserved and what is lost?

This conversion preserves the state dynamics, intervention costs, and strategic interaction. However, it usually does not preserve the costate equations or Hamiltonian stationarity conditions unless these are explicitly added to the learning objective. Therefore, a DRL solver for the induced MDP is not automatically a solver for the PMP boundary-value problem. It is a feedback-policy solver for a discretized control/game problem.

7. PMP and FBSM: Classical Continuous-Time Solution

7.1 PMP conditions

For

$$\min_u \Phi(x(T)) + \int_0^T L(x,u,t)\,\mathrm{d}t, \qquad \dot{x} = f(x,u,t), \tag{67}$$

define the Hamiltonian

$$H(x,u,\lambda,t) = L(x,u,t) + \lambda^\top f(x,u,t). \tag{68}$$

PMP gives [1, 2]

$$\dot{x}^* = H_\lambda(x^*,u^*,\lambda,t), \tag{69}$$

$$\dot{\lambda} = -H_x(x^*,u^*,\lambda,t), \tag{70}$$

$$u^*(t) = \arg\min_{u\in\mathcal{U}} H(x^*,u,\lambda,t), \tag{71}$$

$$\lambda(T) = \Phi_x(x^*(T)). \tag{72}$$

For a zero-sum differential game, the saddle-point condition is

$$u_D^* = \arg\min_{u_D} H(x,u_A^*,u_D,\lambda,t), \qquad u_A^* = \arg\max_{u_A} H(x,u_A,u_D^*,\lambda,t). \tag{73}$$

7.2 Malware example: deriving the control formula

For the controlled malware model, the Hamiltonian is

$$\begin{aligned} H =& A_I I + \frac{B_p}{2}u_p^2 + \frac{B_c}{2}u_c^2 + \lambda_S(-\beta SI - u_p S + \omega R) \\ &+ \lambda_I(\beta SI - \gamma I - u_c I) + \lambda_R(\gamma I + u_p S + u_c I - \omega R). \end{aligned} \tag{74}$$

The stationarity conditions are

$$H_{u_p} = B_p u_p - \lambda_S S + \lambda_R S = 0, \tag{75}$$

$$H_{u_c} = B_c u_c - \lambda_I I + \lambda_R I = 0. \tag{76}$$

With bounds, the controls are

$$u_p^*(t) = \text{clip}\left(\frac{(\lambda_S - \lambda_R)S}{B_p}, 0, u_{p,\max}\right), \tag{77}$$

$$u_c^*(t) = \text{clip}\left(\frac{(\lambda_I - \lambda_R)I}{B_c}, 0, u_{c,\max}\right). \tag{78}$$

This formula is interpretable: if the marginal benefit of moving vulnerable devices to protected devices is high, u_p increases; if the marginal benefit of cleaning compromised devices is high, u_c increases.

7.3 Forward-backward sweep method

The PMP system is a two-point boundary-value problem. FBSM solves it iteratively [3]:

> Guess $u^{(0)}(t) \to$ forward solve $x^{(i)}(t) \to$ backward solve $\lambda^{(i)}(t) \to$ update $u^{(i+1)}(t) \to$ repeat.

A relaxed update is

$$u^{(i+1)}(t) = \alpha\tilde{u}^{(i+1)}(t) + (1-\alpha)u^{(i)}(t), \qquad 0 < \alpha \leq 1. \tag{79}$$

FBSM is an excellent baseline when the dynamics are known and the control is continuous. It becomes less natural when the defender must choose discrete actions such as isolate or deceive.

7.4 FBSM versus RL/DRL: what changes?

FBSM and RL solve different computational versions of the same modeling idea. FBSM normally returns an open-loop control $u^*(t)$ for a fixed initial condition and fixed parameters. If the actual cyber state deviates from the predicted trajectory, the open-loop schedule may be suboptimal unless the problem is solved again. RL/DRL instead learns a feedback policy $\pi(a|s)$ that reacts to the observed state.

Aspect	FBSM/PMP	RL/DRL/MARL
Control type	naturally continuous	discrete, continuous, or hybrid
Output	open-loop trajectory $u(t)$	feedback policy $\pi(a\|s)$
Theory	strong optimality interpretation under assumptions	approximate, data/simulation-driven
Need derivatives	yes, H_x, H_u	no analytic derivatives needed
Adaptation	requires re-solving or feedback extension	policy directly reacts to state
Discrete modes	awkward unless switching/impulse control is used	natural
Training cost	usually cheap for low-dimensional ODEs	can be expensive due to many episodes
Interpretability	high	medium to low, unless policy is analyzed

A useful experimental design is to treat FBSM as a continuous-control benchmark and DRL as a practical feedback controller. If DRL is claimed to improve FBSM, explain whether the improvement comes from

feedback adaptation, discrete action modeling, robustness to stochasticity, or simply from optimizing a different objective.

8. How PMP Should Be Connected to Learning

8.1 Five levels of coupling

Level	Name	Meaning
0	Disconnected learning	PMP is derived, but RL/DRL ignores H, λ, and H_u. It only uses simulator rewards.
1	PMP-formulated RL/-DRL	PMP/differential-game theory defines the continuous-time model and payoff; RL solves the induced MDP or Markov game. This is common in DDQN malware and PPO deception studies [21, 22, 23].
2	PMP-assisted learning	FBSM/PMP controls are used as expert demonstrations, warm starts, reward shaping, or baselines.
3	PMP-informed PIN-N/PIDL	The neural loss explicitly contains state residuals, costate residuals, stationarity residuals $\|H_u\|^2$, and boundary residuals.
4	HJI/Pontryagin operator learning	The learner approximates value gradients or costate operators across states and parameters, as in Pontryagin neural operator work [26].

Writing advice. If a method trains on (s, a, r, s') only, call it a DRL solver for the induced MDP or Markov game. If the loss contains $\dot{\lambda} + H_x$, H_u, or HJI residuals, call it PMP-informed or HJI-informed. This avoids overstating the connection between PMP and learning.

8.2 Why recent papers often use Level 1

PMP/FBSM gives structure but may be too slow, too offline, or too continuous for operational cyber actions. For example, DDQN malware suppression formulates the malware game and then uses DDQN because actions such as patching decisions are discrete [21]. Time-delay deception deployment formulates a differential game but converts it to a Markov game and uses PPO to learn real-time feedback policies [22]. These are valid designs, but the methodological contribution is not "solving PMP by DRL." It is "using DRL to solve the feedback decision problem induced by a differential game."

9. RL and DRL in Cyber Control

This section keeps the main ideas concise. Appendix C gives the prerequisite details.

9.1 Why use RL?

RL is useful when the defender can simulate the cyber system but cannot easily solve the analytic optimality system. It is also useful when actions are operational and discrete. In the MDP derived in

Section 3, RL learns

$$\pi(a|s) \quad \text{or} \quad Q(s,a), \tag{80}$$

so the defender can react to the current cyber state instead of following a precomputed open-loop control $u(t)$.

9.2 DQN for discrete cyber actions

For discrete actions such as no-op, patch, monitor, isolate, and deceive, DQN approximates $Q(s,a)$ using a neural network [5]. The Bellman target is

$$y = r + \gamma_d \max_{a'} Q_{\theta^-}(s', a'), \tag{81}$$

and the training loss is

$$\mathcal{L}_{DQN}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[(Q_\theta(s,a) - y)^2\right]. \tag{82}$$

Double DQN reduces overestimation by using the online network to select the next action and the target network to evaluate it [6]:

$$y^{DDQN} = r + \gamma_d Q_{\theta^-}\left(s', \arg\max_{a'} Q_\theta(s', a')\right). \tag{83}$$

DQN implementation for malware defense.

1. State: $s_k = [S_k, I_k, R_k, z_k, b_k]$, normalized to $[0,1]$.
2. Actions: {no-op, patch, monitor, isolate, deceive}.
3. Transition: map the action to ODE parameters, integrate from t_k to t_{k+1}.
4. Reward: $r_k = -\Delta t_k(A_I I_{k+1} + c_a(a_k) + c_{iso}\mathbf{1}_{a_k=\text{isolate}} I_{k+1})$.
5. Network: MLP $5 \to 64 \to 64 \to 5$.
6. Training: replay buffer, target network, ϵ-greedy exploration, Adam optimizer.

9.3 Actor-critic methods for continuous controls

If actions are continuous intensities such as $u_p, u_c, u_d \in [0,1]$, DQN is inconvenient. Actor-critic methods learn a policy network and a value or Q network. DDPG learns a deterministic actor for continuous actions [7]; PPO uses a clipped policy-gradient objective for stable updates [8]; SAC adds entropy regularization for exploration [9]. A typical actor for cyber defense is

$$\pi_\theta(s) = \left[u_p(s), u_c(s), u_d(s)\right], \qquad u_i(s) = \text{sigmoid}(h_i(s)). \tag{84}$$

9.4 DQN training loop in detail

DQN learns from transitions rather than complete analytic equations. For an ODE-based malware environment, each transition is generated by solving the ODE for one decision interval. The training loop is:

1. Initialize the online Q-network Q_θ and target network Q_{θ^-}.
2. At state s_k, choose a random action with probability ϵ, otherwise choose $a_k = \arg\max_a Q_\theta(s_k, a)$.

3. Run the ODE environment to obtain $(s_{k+1}, r_k, done)$.
4. Store $(s_k, a_k, r_k, s_{k+1}, done)$ in replay buffer $\mathcal{D}$.
5. Sample a mini-batch from $\mathcal{D}$ and compute Bellman targets.
6. Update θ by minimizing squared Bellman error.
7. Periodically copy θ to θ^- or use a soft update.

Typical starting hyperparameters are replay buffer size 10^4–10^6, mini-batch size 64–256, learning rate 10^{-4}–10^{-3}, target update every 100–1000 gradient steps, and an ϵ schedule decreasing from 1 to 0.05. These values should be reported because they affect reproducibility.

9.5 When to use DQN, PPO, SAC, or DDPG

- Use **DQN/DDQN** when actions are a small set of discrete interventions such as patch, isolate, monitor, and deceive.
- Use **PPO** when actions are discrete, continuous, or hybrid and stable policy updates are more important than sample efficiency.
- Use **DDPG/TD3/SAC** when controls are continuous resource intensities. SAC is often more robust because entropy encourages exploration.
- Use **parameterized-action RL** when a decision has both a mode and a continuous intensity, for example patch at intensity 0.7.

A good paper should justify the algorithm from the action space and cyber decision structure, not just use a popular DRL method.

9.6 Reward shaping without changing the problem too much

Reward shaping is useful but dangerous. The base reward should be the negative running cost from the original optimal-control problem:

$$r_k^{base} \approx -\int_{t_k}^{t_{k+1}} L(x(t), a_k)\,\mathrm{d}t. \tag{85}$$

Additional terms can encourage containment or safe exploration, for example

$$r_k = r_k^{base} + \eta_1(C_k - C_{k+1}) - \eta_2 \mathbf{1}\{C_{k+1} > C_{max}\}. \tag{86}$$

However, if the shaping terms dominate, the DRL agent may solve a different problem. In research writing, list the base objective first and state clearly which shaping terms are added for training stability.

10. MARL for Attacker-Defender Interaction

10.1 Markov game implementation

A Markov game is used when the attacker adapts. At each step,

$$a_A^k \sim \pi_A(\cdot|s_k), \qquad a_D^k \sim \pi_D(\cdot|s_k), \tag{87}$$

and the ODE environment returns s_{k+1}, r_A^k, r_D^k. The defender and attacker update their policies using sampled trajectories.

MARL interaction loop

state $s_k \to$ attacker action a_A^k and defender action $a_D^k \to$ ODE cyber transition $s_{k+1} \to$ rewards $r_A^k, r_D^k \to$ update π_A, π_D.

10.2 Concrete attacker-defender malware/deception game

Use $s_k = [V_k, C_k, P_k]$. Defender actions are patch, monitor, isolate, deceive. Attacker actions are scan, exploit, lateral movement, stealth. Deception is modeled as an action effect over the current decision interval, not as a stored population compartment:

$$\beta_k^{\text{eff}} = \beta_k(1 - \chi\eta_k).$$

The ODE between decision points is

$$\dot{V} = -\beta_k^{\text{eff}} VC - \rho_k V + \omega P, \tag{88}$$

$$\dot{C} = \beta_k^{\text{eff}} VC - (\gamma + \mu_k + \iota_k)C, \tag{89}$$

$$\dot{P} = \rho_k V + (\gamma + \mu_k + \iota_k)C - \omega P. \tag{90}$$

The defender reward is

$$r_D^k = -\Delta t_k \left(w_C C_{k+1} + c_\rho \rho_k^2 + c_\mu \mu_k^2 + c_\iota \iota_k^2 + c_\eta \eta_k^2 + c_U \iota_k C_{k+1} \right), \tag{91}$$

and the attacker reward is

$$r_A^k = \Delta t_k \left(q_C C_{k+1} - d_A(a_A^k) - q_P P_{k+1} \right). \tag{92}$$

10.3 Training order

A stable training order is:

1. train defender against scripted attacker;
2. train attacker against fixed or pretrained defender;
3. start self-play from pretrained policies;
4. evaluate against unseen attackers and unseen defense costs.

Independent learning is simple but unstable. Centralized training and decentralized execution (CTDE) improves stability by using critics such as

$$Q_D(s, a_A, a_D), \qquad Q_A(s, a_A, a_D), \tag{93}$$

while actors execute using local observations. MADDPG and MAPPO are representative CTDE-style methods [11, 12].

10.4 Independent learning versus CTDE

In independent learning, the attacker treats the defender as part of the environment and the defender treats the attacker as part of the environment. This is simple but can be unstable because each agent's learning changes the transition distribution for the other agent. CTDE addresses this by allowing the critic to see joint information during training:

$$Q_D(s, a_A, a_D), \qquad Q_A(s, a_A, a_D), \tag{94}$$

while each actor still executes using its own observation. This is appropriate for cyber research because offline training may use complete simulator information, while deployment only uses observable alerts and measurements.

10.5 Self-play curriculum for cyber games

A practical MARL curriculum is:

1. train the defender against a weak scripted attacker;
2. train the defender against stronger scripted attackers;
3. freeze defender and train attacker to exploit it;
4. alternate attacker and defender updates;
5. evaluate against held-out attackers and randomized parameters.

This curriculum is often more stable than starting from fully random self-play. It also produces interpretable experimental stages: basic containment, adaptive attack, and robust defense.

10.6 MADRL algorithm families for attacker-defender games

MARL is the broad field of reinforcement learning with multiple agents. MADRL is the deep-learning version, where policies, critics, or value functions are neural networks. For cyber differential games, four families are especially useful.

Family	How it works	When it is suitable
Independent DQN/PPO	Each agent treats the other agents as part of the environment and learns its own value or policy.	Simple baseline. Useful for small cyber games, but training can be unstable because the environment is nonstationary from each agent's view.
Self-play	Agents repeatedly train against recent or historical versions of opponents.	Good for attacker-defender cyber games where we want adaptive behavior rather than a policy overfitted to one fixed attacker.
Centralized critic, decentralized actors	During training, the critic sees joint actions and possibly global state; during execution, each actor uses local observation.	Useful for simultaneous attacker-defender decisions. MADDPG and MAPPO follow this spirit [11, 12].

Opponent modeling / population training	Agents learn or sample from a population of opponent policies.	Useful when the defender must be robust against many attacker types, not just the latest self-play opponent.

A common CTDE critic for player i is

$$Q_i(s, a_1, \ldots, a_N; \phi_i), \tag{95}$$

while the decentralized actor is

$$\pi_i(a_i|o_i; \theta_i), \tag{96}$$

where o_i may be a local observation rather than the full state. In an APT game, the defender may observe alerts and estimated compromise ratios, while the attacker may observe discovered vulnerable assets. The centralized critic is allowed to use the full simulator state during training because it is a training device, not an operational assumption.

10.7 A step-by-step MADRL recipe for a cyber differential game

For a two-player attacker-defender game, use the following sequence.

1. **Start from the continuous-time game.** Write $\dot{x} = f(x, u_A, u_D)$ and objectives J_A, J_D.
2. **Choose the action schedule.** Choose $0 = t_0 < t_1 < \cdots < t_K = T$. A fixed interval $t_k = k\Delta t$ is convenient, but nonuniform intervals are also valid when operations are event-triggered or risk-triggered.
3. **Define observations.** Use $o_D^k = h_D(x(t_k))$ and $o_A^k = h_A(x(t_k))$. If both agents see the full state, the game is fully observed. If not, the practical model is a partially observed Markov game.
4. **Define actions.** Choose discrete actions, continuous intensities, or hybrid parameterized actions. Example: defender action $a_D = (\text{deceive}, 0.7)$; attacker action $a_A = (\text{lateral movement}, 0.5)$.
5. **Apply impulse first if needed.** If the action is an incident-response impulse, compute $x(t_k^+) = G(x(t_k^-), a_A^k, a_D^k)$.
6. **Integrate the ODE.** Compute $x(t_{k+1}) = \text{ODESolve}(f, x(t_k^+), a_A^k, a_D^k, \Delta t_k)$.
7. **Compute rewards by interval integration.** For player i, set $r_i^k \approx \int_{t_k}^{t_{k+1}} L_i(x, u_A, u_D)\,\mathrm{d}t$ plus any impulse reward/cost.
8. **Train policies.** Use independent PPO for a simple start, then MAPPO or MADDPG for CTDE.
9. **Evaluate game quality.** Report not only average reward but also best-response tests, exploitability proxies, robustness to unseen opponents, and comparison with PMP/FBSM or HODGE-style baselines.

10.8 Equilibrium evaluation in MADRL

A trained pair of policies is not automatically a Nash equilibrium. In a paper, avoid saying that self-play "finds the Nash equilibrium" unless a meaningful equilibrium diagnostic is provided. Practical diagnostics include:

1. **Unilateral deviation test:** freeze the defender and retrain a new attacker best response. Then freeze the attacker and retrain a new defender best response.

2. **Exploitability proxy:** measure how much reward a best-response agent gains over the trained agent.
3. **Opponent pool robustness:** evaluate the defender against random, static, PMP/FBSM, HODGE-style, and learned attackers.
4. **Cost-effectiveness comparison:** compare cumulative security benefit minus control cost, similar in spirit to hybrid-control propaganda-war experiments [18].
5. **Structural sanity check:** verify that actions make cyber sense, e.g., early containment or deception when compromise grows, reduced spending when the system is already safe.

Important wording. For MADRL results, it is safer to write "the learned policy profile is an approximate equilibrium candidate" or "a robust adaptive strategy profile" unless best-response or exploitability analysis supports a stronger Nash claim.

10.9 How PMP/FBSM baselines strengthen MADRL papers

A cyber MADRL paper is stronger when it is not compared only with random or static policies. PMP/FBSM or HODGE-style strategies can serve three roles.

1. **Baseline:** compare learned feedback policies with open-loop continuous-time strategies.
2. **Teacher:** generate state-action pairs from many initial states and pretrain the actor by behavior cloning.
3. **Validation:** evaluate whether learned actions approximately satisfy Hamiltonian stationarity in regions where the action space is continuous.

For example, if PMP gives $u_D^{\mathrm{PMP}}(t)$ for an initial state x_0, simulate small perturbations of x_0, solve FBSM repeatedly, and train a policy $\pi_D(x,t)$ to imitate the resulting controls. Then fine-tune by RL/MADRL. This produces a more transparent bridge between differential-game theory and learning.

11. Neural ODE, PINN, and PIDL: What They Do Differently

11.1 Neural ODE as a dynamics learner

Neural ODE represents continuous dynamics by a neural network [14]:

$$\dot{x} = f_\theta(x,t). \tag{97}$$

For cyber research, the most useful version is often hybrid mechanistic-neural:

$$\dot{x} = f_{known}(x,u,t) + g_\theta(x,u,t). \tag{98}$$

The known term captures malware, APT, or rumor theory; the neural term captures unknown behavior such as attacker adaptation, social reinforcement, hidden vulnerabilities, or platform recommendation effects. Neural ODE learns dynamics; it still needs PMP, MPC, RL, or PINN/PIDL to solve control.

11.2 PINN/PIDL as equation-constrained learning

PINNs train networks to satisfy data and differential equation residuals [15]. Approximate

$$\hat{x}(t) = N_x(t), \qquad \hat{u}(t) = N_u(t), \tag{99}$$

and define

$$r_x(t) = \dot{\hat{x}}(t) - f(\hat{x}(t), \hat{u}(t), t; \theta). \tag{100}$$

A basic loss is

$$\mathcal{L} = \mathcal{L}_{data} + \alpha_f \frac{1}{N_f} \sum_{j=1}^{N_f} \left\| r_x(t_j) \right\|^2 + \alpha_b \mathcal{L}_{boundary}. \tag{101}$$

PIDL is broader than PINN: it can include ODE/PDE residuals, PMP residuals, HJB/HJI residuals, conservation laws, parameter priors, monotonicity constraints, and cyber-specific constraints.

11.3 PMP-informed PINN

A true PMP-informed neural solver approximates state, costate, and control:

$$\hat{x}(t) = N_x(t), \qquad \hat{\lambda}(t) = N_\lambda(t), \qquad \hat{u}(t) = N_u(t). \tag{102}$$

The loss is

$$\mathcal{L}_{PMP} = \frac{1}{N_f} \sum_j \left\| \dot{\hat{x}} - H_\lambda \right\|^2 + \frac{1}{N_f} \sum_j \left\| \dot{\hat{\lambda}} + H_x \right\|^2 \tag{103}$$

$$+ \frac{1}{N_f} \sum_j \left\| H_u \right\|^2 + \mathcal{L}_{IC} + \mathcal{L}_{terminal}. \tag{104}$$

This is different from Level 1 DRL: the Hamiltonian and costate equations appear inside the training objective.

11.4 How Neural ODE is trained in a cyber setting

Suppose observations $x(t_i)$ are available. A Neural ODE predicts

$$\hat{x}(t_{i+1}) = \text{ODESolve}(f_\theta, \hat{x}(t_i), [t_i, t_{i+1}]). \tag{105}$$

The loss is

$$\mathcal{L}_{NODE} = \sum_i \left\| \hat{x}(t_i) - x^{obs}(t_i) \right\|^2. \tag{106}$$

For cyber models, a safer design is not to learn all of f from scratch. Instead use

$$f_\theta(x, u, t) = f_{mechanistic}(x, u, t; \theta_p) + g_\theta(x, u, t), \tag{107}$$

where $f_{mechanistic}$ encodes known propagation and g_θ learns missing mechanisms. This improves interpretability and reduces the amount of data required.

11.5 When PINN/PIDL is preferable to Neural ODE

Neural ODE primarily learns a trajectory-generating vector field from data. PINN/PIDL is preferable when one wants to enforce equations at collocation points, learn parameters under sparse data, impose conservation laws, or solve optimality systems. In short, Neural ODE is a dynamics learner; PINN/PIDL is an equation-constrained learning and solving framework. They can also be combined: Neural ODE learns unknown terms, while PINN/PIDL enforces physics and control residuals.

12. Hybrid Control and Mixed Continuous-Discrete Differential Games

12.1 Why hybrid actions are natural

In practice, a defender chooses both a mode and an intensity:

$$a_D^k = (m_D^k, v_D^k), \quad m_D^k \in \{\text{patch, monitor, isolate, deceive}\}, \quad v_D^k \in [0,1]. \tag{108}$$

The attacker may choose

$$a_A^k = (m_A^k, v_A^k), \quad m_A^k \in \{\text{scan, exploit, lateral, stealth}\}, \quad v_A^k \in [0,1]. \tag{109}$$

Between decision times,

$$\dot{x} = f_{m_A^k, m_D^k}(x, v_A^k, v_D^k, t), \qquad t \in [t_k, t_{k+1}). \tag{110}$$

This is a hybrid differential game: continuous state dynamics, discrete strategic modes, and continuous resource intensities.

12.2 Solution routes

Route 1: mode enumeration plus PMP. Fix a mode sequence, solve continuous intensities by PMP/FBSM, and compare candidate sequences. This is interpretable but grows exponentially with the number of switching times.

Route 2: parameterized action RL. Use two policy heads [13]:

$$\pi(s) = (\pi_m(m|s), \pi_v(v|s,m)). \tag{111}$$

The first head selects a mode; the second outputs the intensity for that mode.

Route 3: hybrid-action MARL. Both attacker and defender use parameterized policies, and a centralized critic evaluates

$$Q(s, m_A, v_A, m_D, v_D). \tag{112}$$

This is appropriate for continuous-discrete cyber games.

Route 4: relaxed PINN/PIDL. Represent discrete modes with soft weights

$$\omega_m(t) = \text{softmax}(g_\theta(t)), \qquad \sum_m \omega_m(t) = 1, \tag{113}$$

and use relaxed dynamics

$$\dot{x} = \sum_m \omega_m(t) f_m(x, v_m, t). \tag{114}$$

After training, select the dominant mode $m^*(t) = \arg\max_m \omega_m(t)$. This is not exact mixed-integer control, but it provides a differentiable approximation.

Route 5: impulse control and impulse games. Some actions, such as emergency isolation or forced password reset, are instantaneous. They can be modeled as

$$\dot{x}(t) = f(x, u, t), \qquad t \neq \tau_j, \tag{115}$$

$$x(\tau_j^+) = G(x(\tau_j^-), a_j). \tag{116}$$

This connects with impulsive APT defense and outsourcing models [19, 20].

12.3 How to choose among hybrid solution routes

The five routes above have different purposes. If the main contribution is mathematical structure, use switching PMP or impulse games. If the main contribution is an operational adaptive defender, use hybrid-action RL or MARL. If the main contribution is data-efficient learning under sparse observations, use PINN/PIDL for parameter learning and then use RL/MARL for the discrete strategic layer. A balanced cyber paper can combine them: PINN/PIDL estimates unknown propagation parameters, PMP/FBSM provides a continuous-control benchmark, and MARL learns hybrid feedback policies under adaptive attackers.

12.4 Example hybrid action mapping

For defender mode m_D and intensity v_D, define

$$\rho_k = \rho_0 + v_D \rho_{max} \mathbf{1}\{m_D = \text{patch}\}, \tag{117}$$

$$\mu_k = \mu_0 + v_D \mu_{max} \mathbf{1}\{m_D = \text{monitor}\}, \tag{118}$$

$$\iota_k = v_D \iota_{max} \mathbf{1}\{m_D = \text{isolate}\}, \tag{119}$$

$$\eta_k = v_D \eta_{max} \mathbf{1}\{m_D = \text{deceive}\}. \tag{120}$$

This map makes the model transparent. It also makes it clear which part is learned: the policy learns m_D and v_D, while the ODE defines how these choices affect cyber dynamics.

13. Compact Case Studies and Experiment Reporting

The earlier sections describe the mathematics. This section is the implementation map: what to build, what to compare, and what result should be reported. It replaces several repetitive recipes with one compact table.

Case	What to build	What the method needs	What to report

FBSM malware baseline	Continuous SIR-style malware ODE with patching or cleaning intensity $u(t)$.	Known parameters, differentiable dynamics, running cost, terminal cost, control bounds.	Control-change convergence, objective, compromised trajectory, peak/final compromise, and open-loop control curve.
DDQN malware defender	Sampled-data MDP with state $s_k = [S, I, R]$, discrete defense modes, jump map, and ODE flow.	Transition simulator, action map, reward, replay buffer, Q-network, fixed or documented nonuniform action schedule.	Training/evaluation return, cumulative compromise, peak/final compromise, defense cost, impulse count, and comparison with no-defense and fixed-policy baselines.
Node-level epidemic-model robustness	Stochastic graph simulator where each node has a local S/I/R state, bursts, and aggregate feedback observation.	Node graph, true propagation parameters, nominal FBSM parameters, trained feedback policy.	A metric where feedback matters: cumulative infected-node exposure under parameter mismatch, peak infected-node share, final infected-node share, cost, and a scaling proxy for node-level PMP/FBSM unknowns.
HODGE-style propaganda game	Hybrid differential game with continuous influence investment and impulsive interventions [18].	Degree-based state, two-player rewards, impulse map, continuous investment costs.	Payoff, cost-effectiveness, impulse timing, lower-bound sensitivity, and approximate Nash or deviation diagnostics.
PMP-informed PINN APT defense	Neural networks for state, costate, and control that satisfy PMP residuals.	APT model, Hamiltonian, boundary conditions, stationarity residual, control bounds.	Residual loss, boundary error, recovered control, comparison with FBSM, and sensitivity to sparse or noisy data.
PINN/PIDL plus MARL pipeline	Estimate unknown parameters first, then train hybrid attacker-defender policies.	Sparse observations, inverse PINN/PIDL model, Markov-game simulator, scripted and learned opponents.	Parameter error, robustness under sampled parameters, game response matrix, unilateral deviation tests, and cyber security metrics.

13.1 Why the node-level epidemic experiment can favor learning

FBSM is a strong baseline when the model is low-dimensional, smooth, known, and solved for the same parameters used in deployment. A feedback learning controller can look better in a different and practically important regime:

- the real system is a node-level S/I/R epidemic graph, so a full PMP formulation would require many state and costate variables;
- the FBSM baseline is open-loop and computed with nominal or underestimated propagation parameters;
- the attacker creates bursts or stochastic shocks that were not present in the nominal trajectory;
- the learned policy observes the current aggregate state and can react immediately at t_k.

This does not mean that DDQN universally dominates FBSM. It means the experiment is measuring *feedback robustness under model mismatch*, not pure optimality for a known continuous-control problem. The metric should therefore be explicit: cumulative infected-node exposure on the node-level epidemic graph, averaged over graph seeds, with the assumed FBSM parameter and true simulator parameter both reported.

13.2 Minimal implementation recipes

FBSM recipe. Create a continuous time mesh, initialize controls, solve the state equation forward, solve the costate equation backward, update controls from stationarity, project onto bounds, relax the update, and stop when the control-change and objective curves settle. The output is an open-loop control curve $u^*(t)$ for a specified initial condition and parameter set.

DDQN or sampled-data RL recipe. Define `step(state, action)` as: observe $x(t_k^-)$, decode the action, apply a true impulse only when the mode is impulsive, integrate the ODE over $[t_k, t_{k+1})$, compute the interval reward, and store $(s_k, a_k, r_k, s_{k+1}, d_k)$ in replay. Zero-order hold is one possible action-to-control map; it should be stated rather than assumed.

CTDE/MARL recipe. Use decentralized actors $\pi_D(a_D|o_D)$ and $\pi_A(a_A|o_A)$ for execution, but centralized critics $Q_D(s, a_D, a_A)$ and $Q_A(s, a_D, a_A)$ during training. Stabilize with scripted-opponent curricula, opponent pools, slower opponent updates, reward normalization, and cross-play evaluation.

13.3 Metrics for the case table

The case table is only useful if the reported metrics match the claim being made. Reward curves alone are not enough:

- model equations, parameter values, horizon, action schedule t_k or Δt_k, solver substeps, observation vector, action map, and reward formula;
- cumulative compromise or misinformation $\int C(t)\,\mathrm{d}t$, peak level, final level, containment time, and total defense cost;
- impulse count and action distribution when hybrid or impulsive controls are used;
- FBSM convergence diagnostics, DDQN/MADRL training curves, and game response or cross-play matrices;
- robustness under parameter perturbation, node-level epidemic graph seeds, or unseen attacker policies;
- computation time for FBSM, DRL inference/training, and PINN/PIDL training when method practicality is part of the claim.

Implementation advice. Start with deterministic no-defense, fixed-patch, fixed-clean, threshold, and random policies before training. Many failed DRL projects fail because the environment, reward, or action mapping is wrong. The learned policy should beat or at least explain its relationship to these simple baselines before stronger claims are made.

14. Concise Experiment Protocol

The case table above describes the algorithms. This shorter protocol is the checklist that should appear in a clean implementation or paper.

Minimal environment contract.

1. `reset()`: set x_0, budget, attack scenario, and decision counter.
2. `observe()`: return $s_k = \psi(x(t_k^-))$.
3. `decode_action(a)`: map a discrete, continuous, or hybrid action into rate controls and possible impulse controls.
4. `jump_map()`: apply $x(t_k^+) = G(x(t_k^-), a_k)$ only for actions that are truly impulsive.
5. `integrate()`: solve the ODE over $[t_k, t_{k+1})$ using the post-jump state and the current interval length Δt_k.
6. `reward()`: approximate the original interval cost and add any impulse cost.

14.1 Concise reporting checklist

Report the model equations, parameter values, action schedule t_k or interval lengths Δt_k, solver substeps, horizon, observation vector, action map, reward formula, random seed, baselines, and metrics. The key cyber metrics are cumulative compromise, peak compromise, final compromise, total defense cost, impulse count, and action distribution. Reward curves alone are not enough.

14.2 Staged implementation

1. Run the deterministic ODE simulator with no defense and fixed defenses.
2. Add impulse and hybrid actions, and verify that the jump map changes $x(t_k^+)$ only when intended.
3. Train a single-agent DDQN defender against scripted attackers.
4. Compare DDQN against no defense, fixed patching, fixed cleaning, and a transparent rule-based baseline.
5. Add attacker learning and evaluate a game matrix or cross-play table, not only a self-play reward curve.
6. Compare FBSM and RL carefully: FBSM is an open-loop continuous-control benchmark for a known model; RL is a sampled-data feedback policy for an induced MDP or Markov game. If learning appears better, state whether the advantage comes from feedback, model mismatch, node-level epidemic scale, discrete actions, or adaptive attackers.

Implementation warning. Most failed cyber DRL experiments fail because the environment or reward is wrong, not because the neural network is too small. Always plot uncontrolled trajectories, fixed-policy trajectories, reward components, and action counts before claiming a learning result.

15. Main Takeaways

PMP and FBSM provide theoretical structure, optimality conditions, and interpretable continuous-control baselines. RL and DRL learn feedback policies for MDPs induced by continuous-time cyber models. MARL extends this to adaptive attackers and defenders through Markov games. Neural ODE learns unknown continuous-time dynamics. PINN and PIDL embed equations, constraints, and possibly PMP or HJI residuals into neural training. Hybrid control is especially important in cybersecurity because real interventions combine discrete modes and continuous intensities.

16. Python Implementation Map and Code Organization

The accompanying code is intentionally organized around the same methodological structure as this note. The mathematical model is separated from the solvers so that FBSM, DDQN, compact CTDE, node-SIPS MAPPO, and shared evaluation metrics can be compared on the same cyber-control timing convention.

Python file	What it demonstrates
`cyber_dynamics.py`	ODE right-hand sides, RK4 integration, and state projection for compartment models.
`cyber_hybrid_env.py`	The central sampled-data hybrid environment: observe, decode action, apply jump map, integrate ODE, compute rewards.
`evaluation_metrics.py`	Shared rollout metrics for policy comparison: cumulative compromise, peak/final compromise, defense cost, impulse count, and action switches.
`fbsm_malware_baseline.py`	Classical PMP/FBSM baseline for continuous malware control.
`ddqn_cyber_defense.py`	Discrete-action DDQN defender against a scripted attacker.
`madrl_ctde_hybrid_game.py`	A compact CTDE attacker-defender Markov-game baseline.
`node_sips_mappo.py`	Community MAPPO on heterogeneous node-SIPS dynamics. Observations include local state, boundary pressure, and known community risk/rate summaries; rewards use criticality-weighted infection and per-node action costs.
`node_level_robustness.py`	Node-level epidemic robustness experiment comparing DDQN aggregate feedback with nominal-parameter FBSM open-loop control.

`run_training_iterations.py`	Produces convergence histories, policy-comparison metrics, game diagnostics, node-level epidemic robustness metrics, and training summaries.
`generate_figures.py`	Rebuilds the README figures, including neural-network and t_k versus τ_j timing diagrams.

Recommended implementation sequence. First run the ODE environment and FBSM baseline. Then train DDQN against a scripted attacker. Only after these two parts are debugged should one move to compact CTDE, node-SIPS MAPPO, or node-level epidemic robustness experiments. Use the metric and figure scripts to compare learned policies against fixed, rule-based, and nominal-parameter optimal-control baselines.

A. Deep Learning Background: Networks, Activations, and Optimization

A feedforward neural network represents a function by composing affine maps and nonlinear activations:

$$h^{(0)} = s, \tag{121}$$

$$h^{(\ell+1)} = \sigma\left(W^{(\ell)}h^{(\ell)} + b^{(\ell)}\right), \tag{122}$$

$$y = W^{(L)}h^{(L)} + b^{(L)}. \tag{123}$$

In cyber-control applications, the input s may be an observation such as $[V, C, P]$ plus contextual features, and the output may be Q-values, action probabilities, continuous controls, or value estimates. The parameters are all matrices and vectors

$$\Theta = \{W^{(\ell)}, b^{(\ell)}\}_{\ell=0}^{L}. \tag{124}$$

Training means minimizing a loss function by gradient-based optimization:

$$\Theta \leftarrow \Theta - \alpha \nabla_{\Theta} \mathcal{L}(\Theta), \tag{125}$$

where α is the learning rate. In practice, Adam is often used before more specialized optimizers.

A.1 Sigmoid, tanh, and softmax

The sigmoid function

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}} \tag{126}$$

maps a real number to $(0, 1)$, which is useful for bounded controls such as patching intensity. If a control has bounds $u \in [u_{\min}, u_{\max}]$, one can set

$$u = u_{\min} + (u_{\max} - u_{\min}) \,\text{sigmoid}(z). \tag{127}$$

The softmax function maps logits ℓ_i to probabilities:

$$p_i = \frac{e^{\ell_i/\tau}}{\sum_{j=1}^{m} e^{\ell_j/\tau}}, \qquad i = 1, \ldots, m. \tag{128}$$

Here $\tau > 0$ is a temperature. Large τ produces smoother probabilities; small τ makes the distribution close to one-hot. Softmax is used in policy networks for discrete actions such as patch, monitor, isolate, and deceive.

A.2 Why scaling matters

Neural networks train more reliably when inputs have similar scales. For example, if $C \in [0, 1]$ but budget is measured in thousands, the network may overemphasize budget. Normalize features before training:

$$\tilde{s}_i = \frac{s_i - \mu_i}{\sigma_i + 10^{-8}}. \tag{129}$$

For compartment models, states are often already in $[0, 1]$, which is convenient.

B. ODE Solver Background for Cyber Control

ODE solvers approximate the solution of

$$\dot{x} = f(x, u, t), \qquad x(t_k) = x_k. \tag{130}$$

In an ODE-RL environment, the solver defines the transition from x_k to x_{k+1}.

B.1 Euler and RK4

The forward Euler method is

$$x_{k+1} = x_k + \Delta t f(x_k, u_k, t_k). \tag{131}$$

It is simple but may be inaccurate or unstable for large Δt. A more reliable fixed-step method is RK4:

$$k_1 = f(x_k, u_k, t_k), \tag{132}$$

$$k_2 = f(x_k + \frac{\Delta t}{2} k_1, u_k, t_k + \frac{\Delta t}{2}), \tag{133}$$

$$k_3 = f(x_k + \frac{\Delta t}{2} k_2, u_k, t_k + \frac{\Delta t}{2}), \tag{134}$$

$$k_4 = f(x_k + \Delta t k_3, u_k, t_k + \Delta t), \tag{135}$$

$$x_{k+1} = x_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{136}$$

RK4 is often a good default for repeated RL training because it is deterministic and easy to reproduce.

B.2 Adaptive solvers

Adaptive solvers select internal step sizes automatically. They are useful for validation but can be slower. When using adaptive solvers in RL, keep the outer action schedule fixed for the episode design, even if the solver uses smaller internal steps. The schedule may be fixed-step or nonuniform, but it should be part of the environment definition rather than an accidental consequence of solver adaptivity.

B.3 Conservation and positivity

Compartment models often require $S + I + R = 1$ and nonnegative states. Numerical solvers may slightly violate these constraints. Use either smaller steps, constrained parameterizations, or projection. Projection

should be reported because it changes the environment transition.

C. MDP, Bellman Equation, and Q-Learning Background

An MDP consists of states s, actions a, transition probabilities $P(s'|s,a)$, rewards $r(s,a,s')$, and discount factor γ_d. The goal is to find a policy $\pi(a|s)$ that maximizes

$$G_k = \sum_{j=0}^{K-k} \gamma_d^j r_{k+j}. \tag{137}$$

The optimal Q-function satisfies the Bellman optimality equation

$$Q^*(s,a) = \mathbb{E}\left[r + \gamma_d \max_{a'} Q^*(s',a') \mid s,a \right]. \tag{138}$$

Classical Q-learning updates a table by

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma_d \max_{a'} Q(s',a') - Q(s,a) \right]. \tag{139}$$

DQN replaces the table by a neural network $Q_\theta(s,a)$ [5].

C.1 DQN and DDQN details

DQN minimizes

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{B}} \left[(y - Q_\theta(s,a))^2 \right], \tag{140}$$

where $\mathcal{B}$ is a replay buffer and

$$y = r + \gamma_d \max_{a'} Q_{\theta^-}(s',a') \tag{141}$$

uses a target network θ^-. Double DQN reduces overestimation by decoupling action selection and evaluation:

$$y = r + \gamma_d Q_{\theta^-}\left(s', \arg\max_{a'} Q_\theta(s',a') \right). \tag{142}$$

In cyber defense, DQN is suitable when actions are discrete, for example no-op, patch, isolate, and deceive.

C.2 Practical DQN hyperparameters

A reasonable first configuration is: two hidden layers with 64 or 128 units, ReLU activations, Adam optimizer, learning rate 10^{-4} to 10^{-3}, replay buffer size 10^4 to 10^5, batch size 64 or 128, and target-network update every 500 to 2000 environment steps. The exploration rate ϵ can decay from 1.0 to 0.05. These are starting points, not universal settings.

D. PPO and Continuous-Action Actor-Critic Background

PPO trains a stochastic policy and a value function. The policy gradient is stabilized by clipping the probability ratio

$$r_t(\phi) = \frac{\pi_\phi(a_t|s_t)}{\pi_{\phi_{old}}(a_t|s_t)}. \tag{143}$$

The clipped objective is

$$\mathcal{L}^{\mathrm{PPO}}(\phi) = \mathbb{E}\left[\min\left(r_t(\phi)\hat{A}_t, \mathrm{clip}(r_t(\phi), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right], \tag{144}$$

where $\hat{A}_t$ is an advantage estimate [8]. PPO is easy to use for both discrete and continuous actions.

D.1 DDPG and SAC

DDPG learns a deterministic actor $a = \mu_\phi(s)$ and a critic $Q_\theta(s, a)$ [7]. It is natural for continuous cyber controls but can be sensitive to exploration and hyperparameters. SAC learns a stochastic policy and maximizes both return and entropy:

$$J(\pi) = \mathbb{E}\left[\sum_k \gamma_d^k \left(r_k + \alpha\mathcal{H}(\pi(\cdot|s_k))\right)\right]. \tag{145}$$

The entropy term encourages exploration, which is useful when the best defense strategy changes over time [9].

E. Markov Game and MARL Background

A two-player Markov game has state s, defender action a_D, attacker action a_A, transition $P(s'|s, a_D, a_A)$, and rewards r_D, r_A [10]. In a zero-sum game,

$$r_A = -r_D. \tag{146}$$

In a general-sum game, the attacker and defender have different objectives.

E.1 Independent learning and nonstationarity

Independent learners treat other agents as part of the environment. This is simple but creates nonstationarity because the opponent's policy changes during training. In cyber games, this can appear as unstable training curves or policies that only work against one attacker.

E.2 Centralized training, decentralized execution

CTDE addresses nonstationarity by using a centralized critic during training:

$$Q_D(s, a_D, a_A), \qquad Q_A(s, a_D, a_A). \tag{147}$$

At execution time, each actor uses only its own observation. This fits cyber defense: training may use full simulator information, but deployment may only use defender observations.

F. Hybrid Actions, Softmax Relaxation, and Action Masks

Hybrid cyber actions combine a discrete mode and a continuous intensity:

$$a = (m, v), \qquad m \in \{1, \ldots, M\}, \quad v \in [0, 1]. \tag{148}$$

A policy can output mode probabilities $p = \mathrm{softmax}(g_m(s))$ and mode-conditioned intensities $v_m = \mathrm{sigmoid}(g_v(s, m))$. The executed action uses one mode m and its corresponding intensity v_m.

F.1 Action masks

Some actions may be infeasible. For example, isolate may be unavailable when budget is too low. Let $M_i(s) \in \{0, 1\}$ be an action mask. A masked softmax can be written as

$$p_i = \frac{M_i(s)e^{\ell_i}}{\sum_j M_j(s)e^{\ell_j}}. \tag{149}$$

This prevents the policy from selecting impossible actions. The mask should represent real operational constraints, not arbitrary tuning.

F.2 Differentiable relaxation

For differentiable training, one can use a soft mixture of modes:

$$u(s) = \sum_{m=1}^{M} p_m(s)u_m(s). \tag{150}$$

This is not the same as executing a single discrete mode, but it is useful in PINN/PIDL or differentiable simulation. If the final system requires discrete actions, validate by converting the soft policy to hard actions.

G. Appendix: Game-Theoretic Background for MARL and MADRL

G.1 Normal-form game, dynamic game, differential game, and Markov game

A normal-form game describes one-shot interaction: each player chooses an action once and receives a payoff. A dynamic game describes repeated or time-evolving interaction. A differential game is a dynamic game in which the state follows a differential equation. A Markov game is a discrete-time feedback version in which the next state depends on the current state and joint actions.

Object	Mathematical form	Learning interpretation
Normal-form game	payoff $R_i(a_1, \ldots, a_N)$	useful for local stage-game analysis
Differential game	$\dot{x} = f(x, u_1, \ldots, u_N)$, $J_i = \int L_i \, \mathrm{d}t$	natural for cyber propagation and resource allocation
Markov game	$s_{k+1} \sim P(\cdot \mid s_k, a_1^k, \ldots, a_N^k)$	the standard environment model for MARL/MADRL
Hybrid Markov game	jump map plus ODE integration between decisions	suitable for patching, incident response, takedown, and propaganda interventions

G.2 Zero-sum and general-sum learning objectives

In a zero-sum game, the defender may solve

$$\max_{\pi_D} \min_{\pi_A} V_D^{\pi_D,\pi_A}(s), \tag{151}$$

or equivalently the attacker maximizes $-V_D$. In a general-sum game, each player has a separate objective:

$$\pi_i^* \in \arg\max_{\pi_i} V_i^{\pi_i,\pi_{-i}^*}(s), \quad i = 1, \ldots, N. \tag{152}$$

Many cyber games are closer to general-sum than pure zero-sum because each side has different operational costs, constraints, and benefits. This is why MADRL papers should state clearly whether they train with one shared zero-sum reward, separate rewards, or shaped rewards.

G.3 Open-loop, feedback, and sampled-data interpretations

Open-loop PMP strategies are functions of time, $u_i(t)$. Feedback strategies are functions of state, $u_i(t, x)$. Sampled-data strategies sit between them: players observe the state at discrete times and then apply a control over the next interval. RL/MADRL most naturally implements sampled-data feedback:

$$a_i^k \sim \pi_i(\cdot|s_k), \quad u_i(t) = \mu_i(a_i^k), \quad t \in [t_k, t_{k+1}). \tag{153}$$

This is often more realistic in cyber settings than ideal continuous feedback because defenders review alerts, budgets, and threat intelligence at decision intervals rather than continuously measuring the entire network state.

G.4 MADDPG-style centralized critic

For continuous or hybrid actions, a centralized critic can be written as

$$Q_i(s, a_1, \ldots, a_N; \phi_i). \tag{154}$$

The target is

$$y_i = r_i + \gamma Q_i^-(s', a_1', \ldots, a_N'), \quad a_j' \sim \pi_j^-(\cdot|o_j'), \tag{155}$$

and the critic minimizes $\mathbb{E}[(Q_i - y_i)^2]$. The actor of player i is updated to increase Q_i while other actions are treated as fixed samples. This is useful when attacker and defender actions are continuous intensities such as attack effort, deception level, or patching intensity.

G.5 MAPPO-style centralized value function

For discrete or mixed actions, MAPPO often uses a centralized value function $V_i(s)$ and decentralized policies $\pi_i(a_i|o_i)$. The clipped PPO objective for each agent is based on

$$\rho_i = \frac{\pi_i(a_i^k|o_i^k)}{\pi_i^{old}(a_i^k|o_i^k)}, \tag{156}$$

$$\mathcal{L}_i^{\text{clip}} = \mathbb{E}\left[\min\left(\rho_i A_i, \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)A_i\right)\right]. \tag{157}$$

For cyber games, MAPPO is often easier to stabilize than MADDPG when actions are discrete or hybrid.

G.6 Common MADRL failure modes in cyber games

1. **Reward domination by cost or damage.** If compromise loss is 1000 times larger than action cost, the learned agent may always choose maximum defense. Normalize terms.
2. **Opponent overfitting.** A defender trained against one scripted attacker may fail against a different attacker. Use opponent pools.
3. **No equilibrium check.** Self-play convergence curves do not prove Nash equilibrium. Use unilateral retraining tests.
4. **Unrealistic observations.** If the defender observes the attacker's hidden state perfectly, the result may be too optimistic.
5. **Ignoring impulse semantics.** If an action is modeled as an impulse, it should create a state jump or an immediate cost; if it only changes a rate over the next interval, call it a discrete decision with continuous flow instead.

H. Appendix: Compact Conversion Checklist

This appendix summarizes the conversion rules without repeating the repository code.

Case	Transition	Learning object
Continuous flow only	$x_{k+1} = \text{ODESolve}(f, x_k, \Gamma(a_k), \Delta t_k)$	MDP if one agent acts; Markov game if multiple agents act.
Impulse plus flow	$y_k = G(x(t_k^-), a_k)$, then $x_{k+1} = \text{ODESolve}(f, y_k, \Gamma(a_k), \Delta t_k)$	Hybrid MDP or hybrid Markov game with jump cost.
Hybrid action	$a_k = (m_k, v_k)$ chooses both mode and intensity	Parameterized-action RL or hybrid-action MARL.
Continuous-time residual learning	Train on ODE/PMP/PINN residuals at sampled collocation points	Not an MDP unless transitions and rewards are explicitly defined.

H.1 Recommended transition order

Use the same order throughout the paper and code:

1. Observe the pre-jump state $x(t_k^-)$.
2. Choose defender and, in games, attacker actions.
3. Apply a jump only if the action is truly impulsive: $x(t_k^+) = G(x(t_k^-), a_k)$.
4. Integrate the ODE flow over $[t_k, t_{k+1})$.
5. Return the next pre-jump observation $x(t_{k+1}^-)$.

H.2 Common mistakes

- Treating every discrete decision as an impulse. A discrete decision may only set ODE rates for the next interval.

- Assuming discretization always means piecewise-constant controls. Zero-order hold is common, but piecewise-linear, event-triggered, and continuous feedback implementations are also possible.
- Confusing solver substeps with learning decision epochs. RK4 substeps improve numerical accuracy; they are not replay-buffer transitions.
- Ignoring the relation between original impulse times τ_j and learning decision times t_k.
- Claiming that DQN solves PMP. DQN solves the sampled-data MDP induced by the simulator unless PMP residuals or PMP expert trajectories are explicitly used.
- Reporting only reward. A convincing cyber-control experiment should also report cumulative compromise, peak compromise, final compromise, defense cost, impulse count, and action distribution.

Where the full implementation lives. The executable implementation is in the repository files `cyber_hybrid_env.py`, `ddqn_cyber_defense.py`, `madrl_ctde_hybrid_game.py`, and `evaluation_metrics.py`. This guide keeps the mathematical mapping and reporting checklist concise.

References

[1] L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko, *The Mathematical Theory of Optimal Processes*. Wiley, 1962.

[2] S. Lenhart and J. T. Workman, *Optimal Control Applied to Biological Models*. Chapman and Hall/CRC, 2007.

[3] M. McAsey, L. Mou, and W. Han, "Convergence of the forward-backward sweep method in optimal control," *Computational Optimization and Applications*, vol. 53, pp. 207–226, 2012.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[5] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[6] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of AAAI*, 2016.

[7] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," arXiv:1509.02971, 2015.

[8] J. Schulman, P. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[9] T. Haarnoja et al., "Soft actor-critic algorithms and applications," arXiv:1812.05905, 2018.

[10] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Proceedings of the 11th International Conference on Machine Learning*, pp. 157–163, 1994.

[11] R. Lowe et al., "Multi-agent actor-critic for mixed cooperative-competitive environments," arXiv:1706.02275, 2017.

[12] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," arXiv:2103.01955, 2021.

[13] M. Hausknecht and P. Stone, "Deep reinforcement learning in parameterized action space,"

arXiv:1511.04143, 2015.

[14] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," *Advances in Neural Information Processing Systems*, 2018.

[15] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[16] J. Sirignano and K. Spiliopoulos, "DGM: A deep learning algorithm for solving partial differential equations," *Journal of Computational Physics*, vol. 375, pp. 1339–1364, 2018.

[17] L.-X. Yang, P. Li, Y. Zhang, X. Yang, Y. Y. Tang, and Y. Xiang, "Effective repair strategy against advanced persistent threat: A differential game approach," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 7, pp. 1713–1728, 2019.

[18] X. Cheng et al., "Cost-effective hybrid control strategies for dynamical propaganda war game," *IEEE Transactions on Information Forensics and Security*, 2024.

[19] H. Sun et al., "Impulsive artificial defense against advanced persistent threat," *IEEE Transactions on Information Forensics and Security*, 2023.

[20] Y. Qin, X. Yang, L.-X. Yang, and K. Huang, "Modeling and study of defense outsourcing against advanced persistent threat through impulsive differential game approach," *Computers & Security*, vol. 145, 104003, 2024.

[21] S. Shen, L. Xie, Y. Zhang, G. Wu, H. Zhang, and S. Yu, "Joint differential game and double deep Q-networks for suppressing malware spread in Industrial Internet of Things," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 5302–5316, 2023.

[22] W. He, J. Tan, R. Wang, Z. Liu, X. Luo, H. Hu, and H. Zhang, "A deep reinforcement learning approach to time delay differential game deception resource deployment," *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 1, pp. 1655–1670, 2026.

[23] W. He, J. Tan, Y. Guo, K. Shang, and H. Zhang, "A deep reinforcement learning-based deception asset selection algorithm in differential games," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 8353–8366, 2024.

[24] S. Shen et al., "Privacy-aware DRL for differential games-assisted malware defense in edge intelligence-enabled Social IoT," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 2680–2695, 2026.

[25] Y. Shen, C. Shepherd, C. M. Ahmed, S. Shen, and S. Yu, "Malware patching strategies in edge intelligence IoT systems: A differential games approach with spiking neural networks," *IEEE Transactions on Dependable and Secure Computing*, early access, 2026.

[26] L. Zhang, M. Ghimire, Z. Xu, W. Zhang, and Y. Ren, "Pontryagin neural operator for solving general-sum differential games with parametric state constraints," in *Proceedings of Learning for Dynamics and Control*, PMLR, vol. 242, pp. 1–13, 2024.

[27] D. Han, S. Jin, and C. Li, "Controlling epidemics with information dynamics: A Neural ODE approach," *IEEE Transactions on Computational Social Systems*, vol. 12, no. 6, pp. 4179–4192, 2025.

[28] Y. Hou et al., "SAPDR: A self-adaptive physics-informed deep learning framework for rumor propagation dynamics," 2026.

[29] Y. Jing et al., "Physics-informed learning for simplicial complex rumor dynamics," 2025.

[30] L. Ma et al., "Physics-informed learning and optimal control for negative information propagation," 2026.

[31] R. Isaacs, *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. Wiley, 1965.

[32] T. Basar and G. J. Olsder, *Dynamic Noncooperative Game Theory*, 2nd ed. SIAM, 1999.